

MODULE 1

Introduction to Deep Learning

1.1 Introduction

Machine learning systems, with shallow or deep architectures, have ability to learn and improve with experience. The process of machine learning begins with the raw data which is used for extracting useful information that helps in decision-making. The primary aim is to allow a machine to learn useful information just like humans do. At abstract level, machine learning can be carried out using following approaches:

Supervised learning adapts a system such that for a given input data it produces a target output. The learning data is made up of tuples (*attributes*, *label*) where “attributes” represent the input data and “label” represents the target output. The goal here is to adapt the system so that for a new input the system can predict the target output. Supervised learning can use both continuous and discrete types of input data.

Unsupervised learning involves data that comprises of input vectors without any target output. There are different objectives in unsupervised learning, such as clustering, density estimation, and visualization. The goal of clustering is to discover groups of similar data items on the basis of measured or perceived similarities between the data items. The purpose of density estimation is to determine the distribution of the data within the input space. In visualization, the data is projected down from a high-dimensional space to two or three dimensions to view the similar data items.

Semi-supervised learning first uses unlabeled data to learn a feature representation of the input data and then uses the learned feature representation to solve the supervised task. The training dataset can be divided into two parts: the data samples with corresponding labels and the data samples where the labels are not known. Semi-supervised learning can involve not providing with an explicit form of error at each time but only a generalized reinforcement is received giving indication of how the system should change its behavior, and this is sometimes referred to as reinforcement

learning. Reinforcement learning has been successful in applications as diverse as autonomous helicopter flight, robot legged locomotion, cell-phone network routing, marketing strategy selection, factory control and efficient webpage indexing.

1.2 Shallow Learning

Shallow architectures are well understood and perform good on many common machine learning problems, and they are still used in a vast majority of today's machine learning applications. However, there has been an increased interest in deep architectures recently, in the hope to find means to solve more complex real-world problems (e.g., image analysis or natural language understanding) for which shallow architectures are unable to learn models adequately.

1.3 Deep Learning

Deep learning is a new area of machine learning which has gained popularity in recent past. Deep learning refers to the architectures which contain multiple hidden layers (deep networks) to learn different features with multiple levels of abstraction. Deep learning algorithms seek to exploit the unknown structure in the input distribution in order to discover good representations, often at multiple levels, with higher level learned features defined in terms of lower level features.

Conventional machine learning techniques are restricted in the way they process the natural data in its raw form. For decades, constructing a pattern recognition or machine learning system required considerable domain expertise and careful hand engineering to come up with a feature extractor that transformed the raw data (such as pixel values of an image) into suitable internal representation or feature vector from which the learning system, such as a classifier, could detect or classify patterns in the input. Deep learning allows inputting the raw data (pixels in case of image data) to the learning algorithm without first extracting features or defining a feature vector. Deep learning algorithms can learn the right set of features, and it does this in a much better way than extracting these features using hand-coding. Instead of handcrafting a set of rules and algorithms to extract features from raw data, deep learning involves learning these features automatically during the training process.

In deep learning, a problem is realized in terms of hierarchy of concepts, with each concept built on the top of the others. The lower layers of the model encode some basic representation of the problem, whereas higher level layers build upon these lower layers to form more complex concepts.

Given an image, the pixel intensity values are fed as inputs to the deep learning system. A number of hidden layers then extract features from the input image. These hidden layers are built upon each other in a hierarchical fashion. At first, the lower level layers of the network detect only edge-like regions. These edge regions are then used

to define corners (where edges intersect) and contours (outlines of objects). The layers in the higher level combine corners and contours to lead to more abstract “object parts” in the next layer. The key aspect of deep learning is that these layers of features are not handcrafted and designed by human engineers; rather, they are learnt from data gradually using a general-purpose learning procedure.

Finally, the output layer classifies the image and obtains the output class label—the output obtained at the output layer is directly influenced by every other node available in the network. This process can be viewed as hierarchical learning as each layer in the network uses the output of previous layers as “building blocks” to construct increasingly more complex concepts at the higher layers. Figure 1.1 compares traditional machine learning approach based on handcrafted features to deep learning approach based on hierarchical representation learning.

Specifically, in deep learning meaningful representations from the input data are learnt by putting emphasis on building complicated mapping using a series of sim-

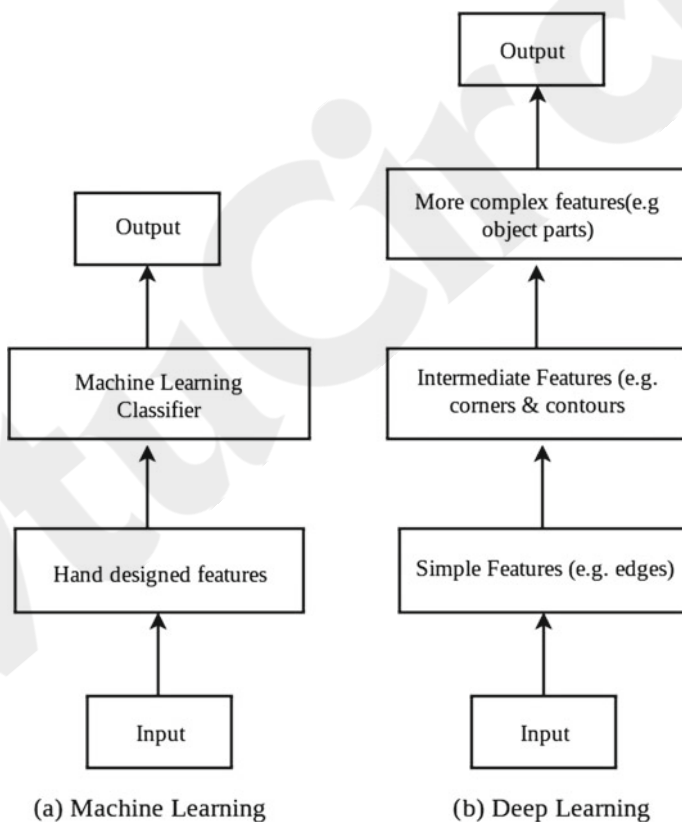


Fig. 1.1 **a** Conventional machine learning using hand-designed feature extraction algorithms **b** deep learning approach using hierarchy of representations that are learnt automatically

ple mappings. The word “deep” refers to learning successive layers of increasingly meaningful representations of input data. The number of layers used to model the data determines the depth of the model. Current deep learning often involves learning tens or even hundreds of successive layers of representation from the training data automatically. The conventional approaches to machine learning often focus on learning only one or two layers of representations of data; such approaches are often categorized as shallow learning. Deep learning and machine learning are sub-fields of Artificial Intelligence (AI). Figure 1.2 illustrates the relationship between AI, machine learning, and deep learning.

In deep learning, the successive layers of representations may be learned via sub-models, which are structured in the form of layers stacked on the top of each other. As deep learning network has typically more layers and parameters, it has the potential to represent more complex inputs. Although deep learning has been around since 1980s, it was relatively unpopular for several years as the computational infrastructure (both hardware and software) was not adequate and the available datasets were quite small. With the decline in the popularity of the conventional neural networks, it was only recently that deep networks made a big reappearance by achieving spectacular results in speech recognition and computer vision tasks. Some of the aspects that helped in the evolution of deep networks are listed below:

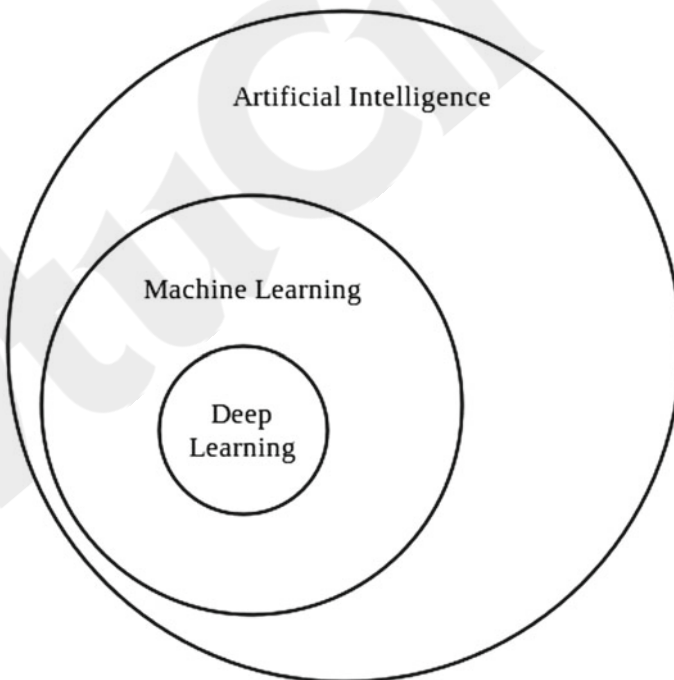


Fig. 1.2 Relationship between AI, machine learning, and deep learning

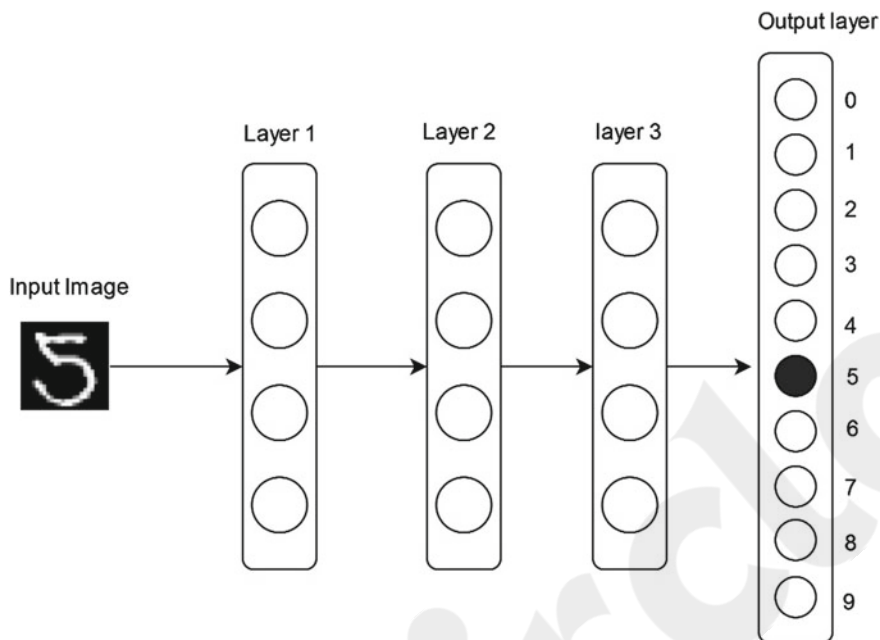


Fig. 1.3 A deep learning network for digit classification

- Improved computational resources for processing massive amounts of data and training much larger models.
- Automatic feature extraction.

The term artificial neural networks has a reference to neuroscience but deep learning networks are not models of the brain; however, deep learning models are formulated by only drawing inspiration from the understanding of biological brain. Not all the components of deep models are inspired by neuroscience; some of them come from empirical exploration, theory, and intuition. The neural activity in our brains is far more complex than might be suggested by simply studying artificial neurons. The learning mechanisms used by deep learning models are in no way comparable to the human brain, but can be described as a mathematical framework for learning representations from data.

Figure 1.3 shows an example of a deep learning architecture that can be used for character recognition. Figure 1.4 shows representations that are learned by the deep learning network. The deep network uses several layers to transform the input image (here a digit) in order to recognize what the digit is. Each layer performs some transformations on the input that it receives from the previous layers.

The deep network transforms the digit image into representations that tend to capture a higher level of abstraction. Each hidden layer transforms the input image into a representation that is increasingly different from the original image and increasingly informative about the final result. The representations learnt help to distinguish

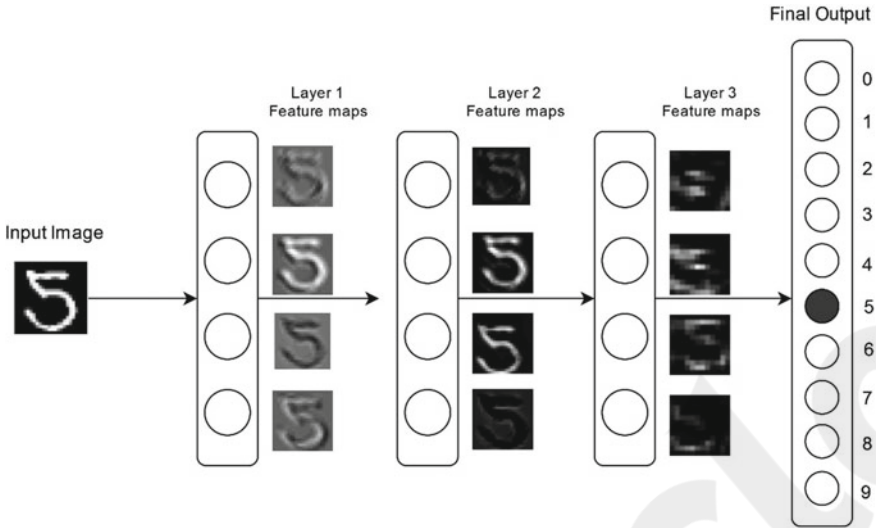


Fig. 1.4 Representations learnt by a deep network for digit classification during the first pass. Network structural changes can be incorporated that result in desired representations at various layers

between different concepts which in turn help to find out similarities between it. Deep network can be thought of as a multistage distillation information operation, where layers use multiple filters on the information to obtain an increasingly transformed form of information (i.e., the information useful with regard to some task).

In summary, a deep learning network constructs features at multiple levels, with higher features constructed as functions of lower ones. It is a fast-growing field that circumvents the problem of feature extraction which is used as a prelude by conventional machine learning approaches. Deep learning is capable of learning the appropriate features by itself, requiring little steering by the user.

1.4 Why to Use Deep Learning

The choice of features that represent a given dataset has a profound impact on the success of a machine learning system. Better results cannot be achieved without identifying which aspects of the problem need to be included for feature extraction that would be more useful to the machine learning algorithm. This requires a machine learning expert to collaborate with the domain expert in order to obtain a useful feature set. A biological brain can easily determine which aspects of the problem it needs to focus on with comparatively little guidance. This is not the case with the artificial agents, thereby making it difficult to create computer learning systems that can respond to high-dimensional input and perform hard AI tasks.

Machine learning practitioners have spent a huge time to extract informative features from the data. At the time of Big Bang introduction of deep learning, the state-of-the-art machine learning algorithms had already took decades of human effort to accumulate relevant set of features required to classify the input.

Deep learning has surpassed those conventional algorithms in accuracy as the features are learnt from the data using a general-purpose learning procedure instead of being designed by human engineers. Deep networks have demonstrated dramatic improvements in computer vision and have dramatically improved machine translation, and have taken off as an effective AI technique that has the ability to recognize spoken words nearly as good as a person can. It has achieved not only the excellent accuracy in machine learning modeling, but it has also demonstrated outstanding generalization power that has even attracted scientists from other academic disciplines. It is now being used as a guide to make key decisions in fields like medicine, finance, manufacturing, and beyond.

Deep learning grew to prominence in 2007, with promising results on perceptual problems such as hearing and seeing problems that humans are very good at, but have long been subtle for the machines. It has enabled the computer scientists to harness the vast computational power and use large volumes of data—audio, video, to teach computers how to do things that seem natural and intuitive for humans, such as spotting objects in the photos, recognizing words or sentences, and translating a document into other language. It has made it possible for machines to output the transcript from an audio clip—speech recognition, to identify whether a mail is spam or not, likelihood of whether a customer will repay his loan and so on; as long as there is enough data to train machines, the possibilities are endless. It has achieved state-of-the-art results on many applications, such as natural language parsing, language modeling, image and character recognition, playing the challenging game of Go, pixels-to-controls video game playing, and in other applications.

Today, many tech giant companies—Facebook, Baidu, Amazon, Microsoft, and Google—have commercially deployed deep learning applications. These companies have vast amount of data and deep learning works well whenever there are vast volumes of data and complex problems to solve. Many companies are using deep learning to develop more helpful and realistic customer service representatives—Chatbots.

In particular, deep learning has made good impact in historically difficult areas of machine learning:

- Near-human-level image classification;
- Near-human-level speech recognition;
- Near-human-level handwriting transcription;
- Improved self-driving cars;
- Digital assistants such as Google Now, Microsoft Cortana, Apple's Siri, and Amazon Alexa;
- Improved ad targeting, as used by Google, Baidu, and Bing;
- Improved search results on the web;
- Ability to answer natural language questions; and
- Superhuman Go, Shogi, and Chess playing.

The exceptional performance of deep models can be mainly attributed to their flexibility in representing a rich set of highly nonlinear functions as well as the devised methods for efficient training of these powerful networks. Furthermore, employing various regularization techniques ensured that deep models with huge numbers of free parameters are statistically desirable in the sense that they will generalize well to unseen data. The automatic and generic approach of feature learning in deep models enables one to use them across different applications (e.g., image classification, speech recognition, language modeling, and information retrieval) with relatively little adjustments. Therefore, deep models seem to be domain-oblivious in the sense that in order to use it across different applications, only a small amount of domain-specific customizations is required. Ideally, the domain-obliviousness of deep networks is advantageous, as having access to a universal and generic model reduces the hassles of adapting for new applications.

Deep learning is still in its infancy, but it is likely that deep learning will have many successes in the near future as it requires little hand engineering and thus can take advantage of vast amount of data and computation power. Deep learning has succeeded in previously unsolved problems which were quite difficult to resolve using machine learning as well as other shallow networks. The dramatic progress of deep learning has sparked such a burst of activity that venture capitalists who did not even know what deep learning was all about some years back, today are suspicious of the startups that do not have it. In near future, deep learning may herald an age where it may assist humans in software development, science, and many more. Integrating deep learning with the whole toolbox of other artificial intelligence techniques may accomplish startling things that will have great impact in the field of technology.

1.5 How Deep Learning Works

Deep networks map input to target via a sequence of layered transformations, and that these layered transformations are learned by exposure to the training examples.

The transformations that a layer applies to its input are determined by the layer's weights, which are basically a bunch of numbers. In other words, transformations implemented by a layer are parameterized by its weights.

In this context, learning can be defined as the process of finding the values of the weights of all layers in the network in such a manner that input examples can be correctly mapped to their associated targets. A deep learning network contains thousands of parameters, and finding the right values of these parameters is not an easy task, particularly when the value of one parameter has an impact on the value of another parameter.

In order to train a deep network one needs to find out how far the calculated output of the network is from the desired value. This measure is obtained by using a loss function, also called as objective function. The objective function calculates

the difference between the predicted output obtained from the network and the true target value for a specific example. This gives a measure of how well the network has learnt a specific example. The objective of the training is to find the values for the weights that minimize the chosen error function.

The difference obtained is then used as a feedback signal to adjust the weights of the network, in a way that loss score for the current example is lowered. This adjustment is done by the optimizer—backpropagation algorithm, the central algorithm in deep learning.

Backpropagation algorithm involves assigning random values to the weight vectors initially, so that the network just implements a series of random transformations. Initially, the output obtained from the network can be far from what it should be, and accordingly the loss score may be very high. With every example that is fed to the network, the weights are adjusted in such a direction that makes the loss score to decrease. This process is repeated a number of times, until the weight values that minimize the loss function are obtained. A network is said to have learned when the output values obtained from the network are as close as they can be to the target values.

1.6 Deep Learning Challenges

Deep learning networks have brought their own set of problems and challenges which outweighed the benefits of deep architectures for several decades. Training these architectures for general use was impractically slow. With limited computational power, deep learning networks were already overtaken by other approaches such as kernel methods. With the significant growth in computational power (particularly in GPUs and distributed computing) and access to large labeled datasets paved the way for its return.

However, despite the remarkable advances in this area, training deep models with a huge number of free parameters is an intricate and ill-posed optimization problem. Many research works have been dedicated to creating efficient training methods for deep architectures. The strategies reported in the literature that deal with the difficulties of training deep networks include developing better optimizers, using well-designed initialization strategies, using activation functions based on local competition and using skip connections between layers with the aim to improve the flow of information. However, deep network training still faces problems which are caused by the stacking of several nonlinear transformations and need to be addressed.

Moreover, deep learning involves using large amounts of data to learn progressively. While large amounts of data are available in many applications, however, in some areas copious amount of data are rarely available. More flexible models are required to achieve an enhanced learning ability when only a limited amount of data is available.

Deep learning networks are very good at solving one problem; however, using deep networks to solve a very similar problem requires retraining and reassessment. Although there are many advancements in this aspect, more work is required in developing deep learning models which can perform multitasks without the need of reworking on the whole architecture.

Optimization for Training Deep Models

Deep learning algorithms involve optimization in many contexts. For example, performing inference in models such as PCA involves solving an optimization problem. We often use analytical optimization to write proofs or design algorithms. Of all of the many optimization problems involved in deep learning, the most difficult is neural network training. It is quite common to invest days to months of time on hundreds of machines in order to solve even a single instance of the neural network training problem. Because this problem is so important and so expensive, a specialized set of optimization techniques have been developed for solving it. This chapter presents these optimization techniques for neural network training.

If you are unfamiliar with the basic principles of gradient-based optimization, we suggest reviewing chapter 4. That chapter includes a brief overview of numerical optimization in general.

This chapter focuses on one particular case of optimization: finding the parameters θ of a neural network that significantly reduce a cost function $J(\theta)$, which typically includes a performance measure evaluated on the entire training set as well as additional regularization terms.

We begin with a description of how optimization used as a training algorithm for a machine learning task differs from pure optimization. Next, we present several of the concrete challenges that make optimization of neural networks difficult. We then define several practical algorithms, including both optimization algorithms themselves and strategies for initializing the parameters. More advanced algorithms adapt their learning rates during training or leverage information contained in

the second derivatives of the cost function. Finally, we conclude with a review of several optimization strategies that are formed by combining simple optimization algorithms into higher-level procedures.

8.1 How Learning Differs from Pure Optimization

Optimization algorithms used for training of deep models differ from traditional optimization algorithms in several ways. Machine learning usually acts indirectly. In most machine learning scenarios, we care about some performance measure P , that is defined with respect to the test set and may also be intractable. We therefore optimize P only indirectly. We reduce a different cost function $J(\boldsymbol{\theta})$ in the hope that doing so will improve P . This is in contrast to pure optimization, where minimizing J is a goal in and of itself. Optimization algorithms for training deep models also typically include some specialization on the specific structure of machine learning objective functions.

Typically, the cost function can be written as an average over the training set, such as

$$J(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} L(f(\mathbf{x}; \boldsymbol{\theta}), y), \quad (8.1)$$

where L is the per-example loss function, $f(\mathbf{x}; \boldsymbol{\theta})$ is the predicted output when the input is $\mathbf{x}$, $\hat{p}_{\text{data}}$ is the empirical distribution. In the supervised learning case, y is the target output. Throughout this chapter, we develop the unregularized supervised case, where the arguments to L are $f(\mathbf{x}; \boldsymbol{\theta})$ and y . However, it is trivial to extend this development, for example, to include $\boldsymbol{\theta}$ or $\mathbf{x}$ as arguments, or to exclude y as arguments, in order to develop various forms of regularization or unsupervised learning.

Equation 8.1 defines an objective function with respect to the training set. We would usually prefer to minimize the corresponding objective function where the expectation is taken across *the data generating distribution* p_{data} rather than just over the finite training set:

$$J^*(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} L(f(\mathbf{x}; \boldsymbol{\theta}), y). \quad (8.2)$$

8.1.1 Empirical Risk Minimization

The goal of a machine learning algorithm is to reduce the expected generalization error given by equation 8.2. This quantity is known as the **risk**. We emphasize here that the expectation is taken over the true underlying distribution p_{data} . If we knew the true distribution $p_{\text{data}}(\mathbf{x}, y)$, risk minimization would be an optimization task

solvable by an optimization algorithm. However, when we do not know $p_{\text{data}}(\mathbf{x}, y)$ but only have a training set of samples, we have a machine learning problem.

The simplest way to convert a machine learning problem back into an optimization problem is to minimize the expected loss on the training set. This means replacing the true distribution $p(\mathbf{x}, y)$ with the empirical distribution $\hat{p}(\mathbf{x}, y)$ defined by the training set. We now minimize the **empirical risk**

$$\mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}(\mathbf{x}, y)}[L(f(\mathbf{x}; \boldsymbol{\theta}), y)] = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}) \quad (8.3)$$

where m is the number of training examples.

The training process based on minimizing this average training error is known as **empirical risk minimization**. In this setting, machine learning is still very similar to straightforward optimization. Rather than optimizing the risk directly, we optimize the empirical risk, and hope that the risk decreases significantly as well. A variety of theoretical results establish conditions under which the true risk can be expected to decrease by various amounts.

However, empirical risk minimization is prone to overfitting. Models with high capacity can simply memorize the training set. In many cases, empirical risk minimization is not really feasible. The most effective modern optimization algorithms are based on gradient descent, but many useful loss functions, such as 0-1 loss, have no useful derivatives (the derivative is either zero or undefined everywhere). These two problems mean that, in the context of deep learning, we rarely use empirical risk minimization. Instead, we must use a slightly different approach, in which the quantity that we actually optimize is even more different from the quantity that we truly want to optimize.

8.1.2 Surrogate Loss Functions and Early Stopping

Sometimes, the loss function we actually care about (say classification error) is not one that can be optimized efficiently. For example, exactly minimizing expected 0-1 loss is typically intractable (exponential in the input dimension), even for a linear classifier (Marcotte and Savard, 1992). In such situations, one typically optimizes a **surrogate loss function** instead, which acts as a proxy but has advantages. For example, the negative log-likelihood of the correct class is typically used as a surrogate for the 0-1 loss. The negative log-likelihood allows the model to estimate the conditional probability of the classes, given the input, and if the model can do that well, then it can pick the classes that yield the least classification error in expectation.

In some cases, a surrogate loss function actually results in being able to learn more. For example, the test set 0-1 loss often continues to decrease for a long time after the training set 0-1 loss has reached zero, when training using the log-likelihood surrogate. This is because even when the expected 0-1 loss is zero, one can improve the robustness of the classifier by further pushing the classes apart from each other, obtaining a more confident and reliable classifier, thus extracting more information from the training data than would have been possible by simply minimizing the average 0-1 loss on the training set.

A very important difference between optimization in general and optimization as we use it for training algorithms is that training algorithms do not usually halt at a local minimum. Instead, a machine learning algorithm usually minimizes a surrogate loss function but halts when a convergence criterion based on early stopping (section 7.8) is satisfied. Typically the early stopping criterion is based on the true underlying loss function, such as 0-1 loss measured on a validation set, and is designed to cause the algorithm to halt whenever overfitting begins to occur. Training often halts while the surrogate loss function still has large derivatives, which is very different from the pure optimization setting, where an optimization algorithm is considered to have converged when the gradient becomes very small.

8.1.3 Batch and Minibatch Algorithms

One aspect of machine learning algorithms that separates them from general optimization algorithms is that the objective function usually decomposes as a sum over the training examples. Optimization algorithms for machine learning typically compute each update to the parameters based on an expected value of the cost function estimated using only a subset of the terms of the full cost function.

For example, maximum likelihood estimation problems, when viewed in log space, decompose into a sum over each example:

$$\boldsymbol{\theta}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}, y^{(i)}; \boldsymbol{\theta}). \quad (8.4)$$

Maximizing this sum is equivalent to maximizing the expectation over the empirical distribution defined by the training set:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{x}, y; \boldsymbol{\theta}). \quad (8.5)$$

Most of the properties of the objective function J used by most of our optimization algorithms are also expectations over the training set. For example, the

most commonly used property is the gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \nabla_{\theta} \log p_{\text{model}}(\mathbf{x}, y; \theta). \quad (8.6)$$

Computing this expectation exactly is very expensive because it requires evaluating the model on every example in the entire dataset. In practice, we can compute these expectations by randomly sampling a small number of examples from the dataset, then taking the average over only those examples.

Recall that the standard error of the mean (equation 5.46) estimated from n samples is given by $\sigma / \sqrt{n}$, where σ is the true standard deviation of the value of the samples. The denominator of $\sqrt{n}$ shows that there are less than linear returns to using more examples to estimate the gradient. Compare two hypothetical estimates of the gradient, one based on 100 examples and another based on 10,000 examples. The latter requires 100 times more computation than the former, but reduces the standard error of the mean only by a factor of 10. Most optimization algorithms converge much faster (in terms of total computation, not in terms of number of updates) if they are allowed to rapidly compute approximate estimates of the gradient rather than slowly computing the exact gradient.

Another consideration motivating statistical estimation of the gradient from a small number of samples is redundancy in the training set. In the worst case, all m samples in the training set could be identical copies of each other. A sampling-based estimate of the gradient could compute the correct gradient with a single sample, using m times less computation than the naive approach. In practice, we are unlikely to truly encounter this worst-case situation, but we may find large numbers of examples that all make very similar contributions to the gradient.

Optimization algorithms that use the entire training set are called **batch** or **deterministic** gradient methods, because they process all of the training examples simultaneously in a large batch. This terminology can be somewhat confusing because the word “batch” is also often used to describe the minibatch used by minibatch stochastic gradient descent. Typically the term “batch gradient descent” implies the use of the full training set, while the use of the term “batch” to describe a group of examples does not. For example, it is very common to use the term “batch size” to describe the size of a minibatch.

Optimization algorithms that use only a single example at a time are sometimes called **stochastic** or sometimes **online** methods. The term online is usually reserved for the case where the examples are drawn from a stream of continually created examples rather than from a fixed-size training set over which several passes are made.

Most algorithms used for deep learning fall somewhere in between, using more

than one but less than all of the training examples. These were traditionally called **minibatch** or **minibatch stochastic** methods and it is now common to simply call them **stochastic** methods.

The canonical example of a stochastic method is stochastic gradient descent, presented in detail in section 8.3.1.

Minibatch sizes are generally driven by the following factors:

- Larger batches provide a more accurate estimate of the gradient, but with less than linear returns.
- Multicore architectures are usually underutilized by extremely small batches. This motivates using some absolute minimum batch size, below which there is no reduction in the time to process a minibatch.
- If all examples in the batch are to be processed in parallel (as is typically the case), then the amount of memory scales with the batch size. For many hardware setups this is the limiting factor in batch size.
- Some kinds of hardware achieve better runtime with specific sizes of arrays. Especially when using GPUs, it is common for power of 2 batch sizes to offer better runtime. Typical power of 2 batch sizes range from 32 to 256, with 16 sometimes being attempted for large models.
- Small batches can offer a regularizing effect (Wilson and Martinez, 2003), perhaps due to the noise they add to the learning process. Generalization error is often best for a batch size of 1. Training with such a small batch size might require a small learning rate to maintain stability due to the high variance in the estimate of the gradient. The total runtime can be very high due to the need to make more steps, both because of the reduced learning rate and because it takes more steps to observe the entire training set.

Different kinds of algorithms use different kinds of information from the minibatch in different ways. Some algorithms are more sensitive to sampling error than others, either because they use information that is difficult to estimate accurately with few samples, or because they use information in ways that amplify sampling errors more. Methods that compute updates based only on the gradient $\mathbf{g}$ are usually relatively robust and can handle smaller batch sizes like 100. Second-order methods, which use also the Hessian matrix $\mathbf{H}$ and compute updates such as $\mathbf{H}^{-1}\mathbf{g}$, typically require much larger batch sizes like 10,000. These large batch sizes are required to minimize fluctuations in the estimates of $\mathbf{H}^{-1}\mathbf{g}$. Suppose that $\mathbf{H}$ is estimated perfectly but has a poor condition number. Multiplication by

$\mathbf{H}$ or its inverse amplifies pre-existing errors, in this case, estimation errors in $\mathbf{g}$. Very small changes in the estimate of $\mathbf{g}$ can thus cause large changes in the update $\mathbf{H}^{-1}\mathbf{g}$, even if $\mathbf{H}$ were estimated perfectly. Of course, $\mathbf{H}$ will be estimated only approximately, so the update $\mathbf{H}^{-1}\mathbf{g}$ will contain even more error than we would predict from applying a poorly conditioned operation to the estimate of $\mathbf{g}$.

It is also crucial that the minibatches be selected randomly. Computing an unbiased estimate of the expected gradient from a set of samples requires that those samples be independent. We also wish for two subsequent gradient estimates to be independent from each other, so two subsequent minibatches of examples should also be independent from each other. Many datasets are most naturally arranged in a way where successive examples are highly correlated. For example, we might have a dataset of medical data with a long list of blood sample test results. This list might be arranged so that first we have five blood samples taken at different times from the first patient, then we have three blood samples taken from the second patient, then the blood samples from the third patient, and so on. If we were to draw examples in order from this list, then each of our minibatches would be extremely biased, because it would represent primarily one patient out of the many patients in the dataset. In cases such as these where the order of the dataset holds some significance, it is necessary to shuffle the examples before selecting minibatches. For very large datasets, for example datasets containing billions of examples in a data center, it can be impractical to sample examples truly uniformly at random every time we want to construct a minibatch. Fortunately, in practice it is usually sufficient to shuffle the order of the dataset once and then store it in shuffled fashion. This will impose a fixed set of possible minibatches of consecutive examples that all models trained thereafter will use, and each individual model will be forced to reuse this ordering every time it passes through the training data. However, this deviation from true random selection does not seem to have a significant detrimental effect. Failing to ever shuffle the examples in any way can seriously reduce the effectiveness of the algorithm.

Many optimization problems in machine learning decompose over examples well enough that we can compute entire separate updates over different examples in parallel. In other words, we can compute the update that minimizes $J(\mathbf{X})$ for one minibatch of examples $\mathbf{X}$ at the same time that we compute the update for several other minibatches. Such asynchronous parallel distributed approaches are discussed further in section 12.1.3.

An interesting motivation for minibatch stochastic gradient descent is that it follows the gradient of the true *generalization error* (equation 8.2) so long as no examples are repeated. Most implementations of minibatch stochastic gradient

descent shuffle the dataset once and then pass through it multiple times. On the first pass, each minibatch is used to compute an unbiased estimate of the true generalization error. On the second pass, the estimate becomes biased because it is formed by re-sampling values that have already been used, rather than obtaining new fair samples from the data generating distribution.

The fact that stochastic gradient descent minimizes generalization error is easiest to see in the online learning case, where examples or minibatches are drawn from a **stream** of data. In other words, instead of receiving a fixed-size training set, the learner is similar to a living being who sees a new example at each instant, with every example $(\mathbf{x}, y)$ coming from the data generating distribution $p_{\text{data}}(\mathbf{x}, y)$. In this scenario, examples are never repeated; every experience is a fair sample from p_{data} .

The equivalence is easiest to derive when both $\mathbf{x}$ and y are discrete. In this case, the generalization error (equation 8.2) can be written as a sum

$$J^*(\boldsymbol{\theta}) = \sum_{\mathbf{x}} \sum_y p_{\text{data}}(\mathbf{x}, y) L(f(\mathbf{x}; \boldsymbol{\theta}), y), \quad (8.7)$$

with the exact gradient

$$\mathbf{g} = \nabla_{\boldsymbol{\theta}} J^*(\boldsymbol{\theta}) = \sum_{\mathbf{x}} \sum_y p_{\text{data}}(\mathbf{x}, y) \nabla_{\boldsymbol{\theta}} L(f(\mathbf{x}; \boldsymbol{\theta}), y). \quad (8.8)$$

We have already seen the same fact demonstrated for the log-likelihood in equation 8.5 and equation 8.6; we observe now that this holds for other functions L besides the likelihood. A similar result can be derived when $\mathbf{x}$ and y are continuous, under mild assumptions regarding p_{data} and L .

Hence, we can obtain an unbiased estimator of the exact gradient of the generalization error by sampling a minibatch of examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $y^{(i)}$ from the data generating distribution p_{data} , and computing the gradient of the loss with respect to the parameters for that minibatch:

$$\hat{\mathbf{g}} = \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)}). \quad (8.9)$$

Updating $\boldsymbol{\theta}$ in the direction of $\hat{\mathbf{g}}$ performs SGD on the generalization error.

Of course, this interpretation only applies when examples are not reused. Nonetheless, it is usually best to make several passes through the training set, unless the training set is extremely large. When multiple such epochs are used, only the first epoch follows the unbiased gradient of the generalization error, but

of course, the additional epochs usually provide enough benefit due to decreased training error to offset the harm they cause by increasing the gap between training error and test error.

With some datasets growing rapidly in size, faster than computing power, it is becoming more common for machine learning applications to use each training example only once or even to make an incomplete pass through the training set. When using an extremely large training set, overfitting is not an issue, so underfitting and computational efficiency become the predominant concerns. See also [Bottou and Bousquet \(2008\)](#) for a discussion of the effect of computational bottlenecks on generalization error, as the number of training examples grows.

8.2 Challenges in Neural Network Optimization

Optimization in general is an extremely difficult task. Traditionally, machine learning has avoided the difficulty of general optimization by carefully designing the objective function and constraints to ensure that the optimization problem is convex. When training neural networks, we must confront the general non-convex case. Even convex optimization is not without its complications. In this section, we summarize several of the most prominent challenges involved in optimization for training deep models.

8.2.1 Ill-Conditioning

Some challenges arise even when optimizing convex functions. Of these, the most prominent is ill-conditioning of the Hessian matrix $\mathbf{H}$. This is a very general problem in most numerical optimization, convex or otherwise, and is described in more detail in section [4.3.1](#).

The ill-conditioning problem is generally believed to be present in neural network training problems. Ill-conditioning can manifest by causing SGD to get “stuck” in the sense that even very small steps increase the cost function.

Recall from equation [4.9](#) that a second-order Taylor series expansion of the cost function predicts that a gradient descent step of $-\epsilon \mathbf{g}$ will add

$$\frac{1}{2}\epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g} - \epsilon \mathbf{g}^\top \mathbf{g} \quad (8.10)$$

to the cost. Ill-conditioning of the gradient becomes a problem when $\frac{1}{2}\epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g}$ exceeds $\epsilon \mathbf{g}^\top \mathbf{g}$. To determine whether ill-conditioning is detrimental to a neural network training task, one can monitor the squared gradient norm $\mathbf{g}^\top \mathbf{g}$ and

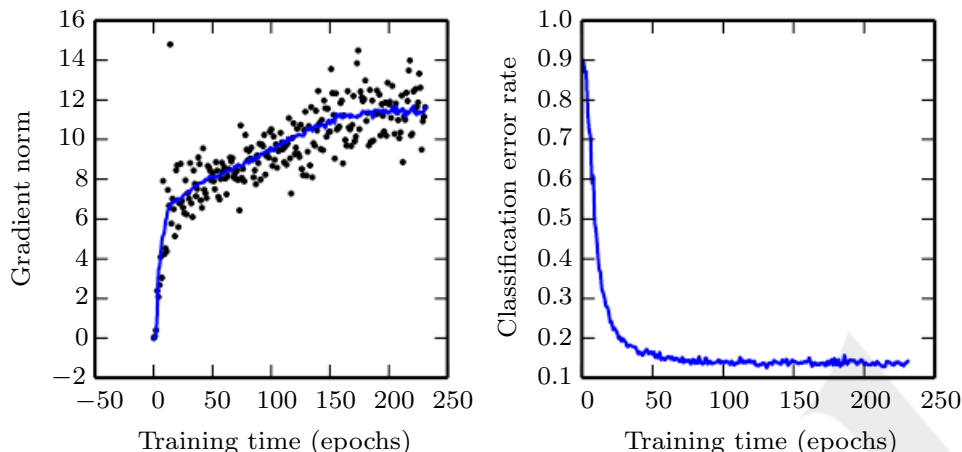


Figure 8.1: Gradient descent often does not arrive at a critical point of any kind. In this example, the gradient norm increases throughout training of a convolutional network used for object detection. *(Left)* A scatterplot showing how the norms of individual gradient evaluations are distributed over time. To improve legibility, only one gradient norm is plotted per epoch. The running average of all gradient norms is plotted as a solid curve. The gradient norm clearly increases over time, rather than decreasing as we would expect if the training process converged to a critical point. *(Right)* Despite the increasing gradient, the training process is reasonably successful. The validation set classification error decreases to a low level.

the $\mathbf{g}^\top \mathbf{H} \mathbf{g}$ term. In many cases, the gradient norm does not shrink significantly throughout learning, but the $\mathbf{g}^\top \mathbf{H} \mathbf{g}$ term grows by more than an order of magnitude. The result is that learning becomes very slow despite the presence of a strong gradient because the learning rate must be shrunk to compensate for even stronger curvature. Figure 8.1 shows an example of the gradient increasing significantly during the successful training of a neural network.

Though ill-conditioning is present in other settings besides neural network training, some of the techniques used to combat it in other contexts are less applicable to neural networks. For example, Newton’s method is an excellent tool for minimizing convex functions with poorly conditioned Hessian matrices, but in the subsequent sections we will argue that Newton’s method requires significant modification before it can be applied to neural networks.

8.2.2 Local Minima

One of the most prominent features of a convex optimization problem is that it can be reduced to the problem of finding a local minimum. Any local minimum is

guaranteed to be a global minimum. Some convex functions have a flat region at the bottom rather than a single global minimum point, but any point within such a flat region is an acceptable solution. When optimizing a convex function, we know that we have reached a good solution if we find a critical point of any kind.

With non-convex functions, such as neural nets, it is possible to have many local minima. Indeed, nearly any deep model is essentially guaranteed to have an extremely large number of local minima. However, as we will see, this is not necessarily a major problem.

Neural networks and any models with multiple equivalently parametrized latent variables all have multiple local minima because of the **model identifiability** problem. A model is said to be identifiable if a sufficiently large training set can rule out all but one setting of the model's parameters. Models with latent variables are often not identifiable because we can obtain equivalent models by exchanging latent variables with each other. For example, we could take a neural network and modify layer 1 by swapping the incoming weight vector for unit i with the incoming weight vector for unit j , then doing the same for the outgoing weight vectors. If we have m layers with n units each, then there are $n!^m$ ways of arranging the hidden units. This kind of non-identifiability is known as **weight space symmetry**.

In addition to weight space symmetry, many kinds of neural networks have additional causes of non-identifiability. For example, in any rectified linear or maxout network, we can scale all of the incoming weights and biases of a unit by α if we also scale all of its outgoing weights by $\frac{1}{\alpha}$. This means that—if the cost function does not include terms such as weight decay that depend directly on the weights rather than the models' outputs—every local minimum of a rectified linear or maxout network lies on an $(m \times n)$ -dimensional hyperbola of equivalent local minima.

These model identifiability issues mean that there can be an extremely large or even uncountably infinite amount of local minima in a neural network cost function. However, all of these local minima arising from non-identifiability are equivalent to each other in cost function value. As a result, these local minima are not a problematic form of non-convexity.

Local minima can be problematic if they have high cost in comparison to the global minimum. One can construct small neural networks, even without hidden units, that have local minima with higher cost than the global minimum (Sontag and Sussman, 1989; Brady *et al.*, 1989; Gori and Tesi, 1992). If local minima with high cost are common, this could pose a serious problem for gradient-based optimization algorithms.

It remains an open question whether there are many local minima of high cost

for networks of practical interest and whether optimization algorithms encounter them. For many years, most practitioners believed that local minima were a common problem plaguing neural network optimization. Today, that does not appear to be the case. The problem remains an active area of research, but experts now suspect that, for sufficiently large neural networks, most local minima have a low cost function value, and that it is not important to find a true global minimum rather than to find a point in parameter space that has low but not minimal cost (Saxe *et al.*, 2013; Dauphin *et al.*, 2014; Goodfellow *et al.*, 2015; Choromanska *et al.*, 2014).

Many practitioners attribute nearly all difficulty with neural network optimization to local minima. We encourage practitioners to carefully test for specific problems. A test that can rule out local minima as the problem is to plot the norm of the gradient over time. If the norm of the gradient does not shrink to insignificant size, the problem is neither local minima nor any other kind of critical point. This kind of negative test can rule out local minima. In high dimensional spaces, it can be very difficult to positively establish that local minima are the problem. Many structures other than local minima also have small gradients.

8.2.3 Plateaus, Saddle Points and Other Flat Regions

For many high-dimensional non-convex functions, local minima (and maxima) are in fact rare compared to another kind of point with zero gradient: a saddle point. Some points around a saddle point have greater cost than the saddle point, while others have a lower cost. At a saddle point, the Hessian matrix has both positive and negative eigenvalues. Points lying along eigenvectors associated with positive eigenvalues have greater cost than the saddle point, while points lying along negative eigenvalues have lower value. We can think of a saddle point as being a local minimum along one cross-section of the cost function and a local maximum along another cross-section. See figure 4.5 for an illustration.

Many classes of random functions exhibit the following behavior: in low-dimensional spaces, local minima are common. In higher dimensional spaces, local minima are rare and saddle points are more common. For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ of this type, the expected ratio of the number of saddle points to local minima grows exponentially with n . To understand the intuition behind this behavior, observe that the Hessian matrix at a local minimum has only positive eigenvalues. The Hessian matrix at a saddle point has a mixture of positive and negative eigenvalues. Imagine that the sign of each eigenvalue is generated by flipping a coin. In a single dimension, it is easy to obtain a local minimum by tossing a coin and getting heads once. In n -dimensional space, it is exponentially unlikely that all n coin tosses will

be heads. See [Dauphin et al. \(2014\)](#) for a review of the relevant theoretical work.

An amazing property of many random functions is that the eigenvalues of the Hessian become more likely to be positive as we reach regions of lower cost. In our coin tossing analogy, this means we are more likely to have our coin come up heads n times if we are at a critical point with low cost. This means that local minima are much more likely to have low cost than high cost. Critical points with high cost are far more likely to be saddle points. Critical points with extremely high cost are more likely to be local maxima.

This happens for many classes of random functions. Does it happen for neural networks? [Baldi and Hornik \(1989\)](#) showed theoretically that shallow autoencoders (feedforward networks trained to copy their input to their output, described in chapter 14) with no nonlinearities have global minima and saddle points but no local minima with higher cost than the global minimum. They observed without proof that these results extend to deeper networks without nonlinearities. The output of such networks is a linear function of their input, but they are useful to study as a model of nonlinear neural networks because their loss function is a non-convex function of their parameters. Such networks are essentially just multiple matrices composed together. [Saxe et al. \(2013\)](#) provided exact solutions to the complete learning dynamics in such networks and showed that learning in these models captures many of the qualitative features observed in the training of deep models with nonlinear activation functions. [Dauphin et al. \(2014\)](#) showed experimentally that real neural networks also have loss functions that contain very many high-cost saddle points. [Choromanska et al. \(2014\)](#) provided additional theoretical arguments, showing that another class of high-dimensional random functions related to neural networks does so as well.

What are the implications of the proliferation of saddle points for training algorithms? For first-order optimization algorithms that use only gradient information, the situation is unclear. The gradient can often become very small near a saddle point. On the other hand, gradient descent empirically seems to be able to escape saddle points in many cases. [Goodfellow et al. \(2015\)](#) provided visualizations of several learning trajectories of state-of-the-art neural networks, with an example given in figure 8.2. These visualizations show a flattening of the cost function near a prominent saddle point where the weights are all zero, but they also show the gradient descent trajectory rapidly escaping this region. [Goodfellow et al. \(2015\)](#) also argue that continuous-time gradient descent may be shown analytically to be repelled from, rather than attracted to, a nearby saddle point, but the situation may be different for more realistic uses of gradient descent.

For Newton's method, it is clear that saddle points constitute a problem.

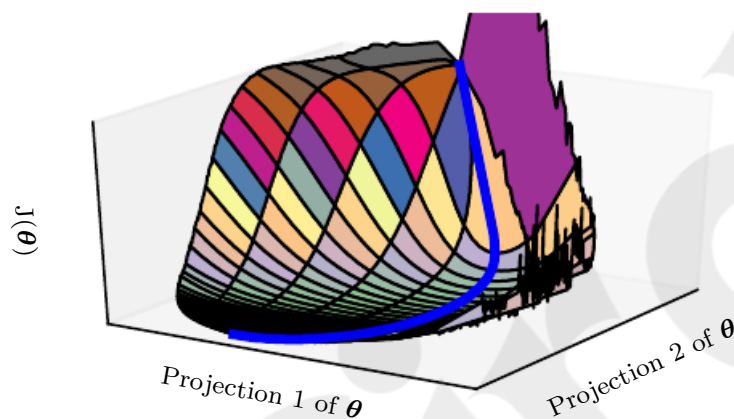


Figure 8.2: A visualization of the cost function of a neural network. Image adapted with permission from [Goodfellow *et al.* \(2015\)](#). These visualizations appear similar for feedforward neural networks, convolutional networks, and recurrent networks applied to real object recognition and natural language processing tasks. Surprisingly, these visualizations usually do not show many conspicuous obstacles. Prior to the success of stochastic gradient descent for training very large models beginning in roughly 2012, neural net cost function surfaces were generally believed to have much more non-convex structure than is revealed by these projections. The primary obstacle revealed by this projection is a saddle point of high cost near where the parameters are initialized, but, as indicated by the blue path, the SGD training trajectory escapes this saddle point readily. Most of training time is spent traversing the relatively flat valley of the cost function, which may be due to high noise in the gradient, poor conditioning of the Hessian matrix in this region, or simply the need to circumnavigate the tall “mountain” visible in the figure via an indirect arcing path.

Gradient descent is designed to move “downhill” and is not explicitly designed to seek a critical point. Newton’s method, however, is designed to solve for a point where the gradient is zero. Without appropriate modification, it can jump to a saddle point. The proliferation of saddle points in high dimensional spaces presumably explains why second-order methods have not succeeded in replacing gradient descent for neural network training. Dauphin *et al.* (2014) introduced a **saddle-free Newton method** for second-order optimization and showed that it improves significantly over the traditional version. Second-order methods remain difficult to scale to large neural networks, but this saddle-free approach holds promise if it could be scaled.

There are other kinds of points with zero gradient besides minima and saddle points. There are also maxima, which are much like saddle points from the perspective of optimization—many algorithms are not attracted to them, but unmodified Newton’s method is. Maxima of many classes of random functions become exponentially rare in high dimensional space, just like minima do.

There may also be wide, flat regions of constant value. In these locations, the gradient and also the Hessian are all zero. Such degenerate locations pose major problems for all numerical optimization algorithms. In a convex problem, a wide, flat region must consist entirely of global minima, but in a general optimization problem, such a region could correspond to a high value of the objective function.

8.2.4 Cliffs and Exploding Gradients

Neural networks with many layers often have extremely steep regions resembling cliffs, as illustrated in figure 8.3. These result from the multiplication of several large weights together. On the face of an extremely steep cliff structure, the gradient update step can move the parameters extremely far, usually jumping off of the cliff structure altogether.

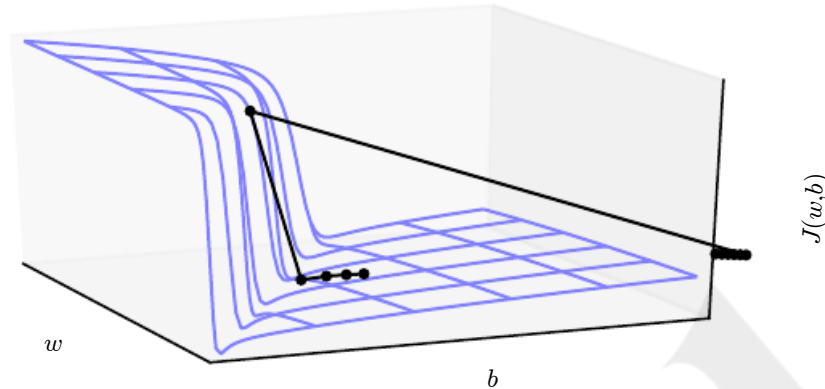


Figure 8.3: The objective function for highly nonlinear deep neural networks or for recurrent neural networks often contains sharp nonlinearities in parameter space resulting from the multiplication of several parameters. These nonlinearities give rise to very high derivatives in some places. When the parameters get close to such a cliff region, a gradient descent update can catapult the parameters very far, possibly losing most of the optimization work that had been done. Figure adapted with permission from [Pascanu et al. \(2013\)](#).

The cliff can be dangerous whether we approach it from above or from below, but fortunately its most serious consequences can be avoided using the **gradient clipping** heuristic described in section 10.11.1. The basic idea is to recall that the gradient does not specify the optimal step size, but only the optimal direction within an infinitesimal region. When the traditional gradient descent algorithm proposes to make a very large step, the gradient clipping heuristic intervenes to reduce the step size to be small enough that it is less likely to go outside the region where the gradient indicates the direction of approximately steepest descent. Cliff structures are most common in the cost functions for recurrent neural networks, because such models involve a multiplication of many factors, with one factor for each time step. Long temporal sequences thus incur an extreme amount of multiplication.

8.2.5 Long-Term Dependencies

Another difficulty that neural network optimization algorithms must overcome arises when the computational graph becomes extremely deep. Feedforward networks with many layers have such deep computational graphs. So do recurrent networks, described in chapter 10, which construct very deep computational graphs

by repeatedly applying the same operation at each time step of a long temporal sequence. Repeated application of the same parameters gives rise to especially pronounced difficulties.

For example, suppose that a computational graph contains a path that consists of repeatedly multiplying by a matrix $\mathbf{W}$. After t steps, this is equivalent to multiplying by $\mathbf{W}^t$. Suppose that $\mathbf{W}$ has an eigendecomposition $\mathbf{W} = \mathbf{V}\text{diag}(\boldsymbol{\lambda})\mathbf{V}^{-1}$. In this simple case, it is straightforward to see that

$$\mathbf{W}^t = (\mathbf{V}\text{diag}(\boldsymbol{\lambda})\mathbf{V}^{-1})^t = \mathbf{V}\text{diag}(\boldsymbol{\lambda})^t\mathbf{V}^{-1}. \quad (8.11)$$

Any eigenvalues λ_i that are not near an absolute value of 1 will either explode if they are greater than 1 in magnitude or vanish if they are less than 1 in magnitude. The **vanishing and exploding gradient problem** refers to the fact that gradients through such a graph are also scaled according to $\text{diag}(\boldsymbol{\lambda})^t$. Vanishing gradients make it difficult to know which direction the parameters should move to improve the cost function, while exploding gradients can make learning unstable. The cliff structures described earlier that motivate gradient clipping are an example of the exploding gradient phenomenon.

The repeated multiplication by $\mathbf{W}$ at each time step described here is very similar to the **power method** algorithm used to find the largest eigenvalue of a matrix $\mathbf{W}$ and the corresponding eigenvector. From this point of view it is not surprising that $\mathbf{x}^\top \mathbf{W}^t$ will eventually discard all components of $\mathbf{x}$ that are orthogonal to the principal eigenvector of $\mathbf{W}$.

Recurrent networks use the same matrix $\mathbf{W}$ at each time step, but feedforward networks do not, so even very deep feedforward networks can largely avoid the vanishing and exploding gradient problem ([Sussillo, 2014](#)).

We defer a further discussion of the challenges of training recurrent networks until section [10.7](#), after recurrent networks have been described in more detail.

8.2.6 Inexact Gradients

Most optimization algorithms are designed with the assumption that we have access to the exact gradient or Hessian matrix. In practice, we usually only have a noisy or even biased estimate of these quantities. Nearly every deep learning algorithm relies on sampling-based estimates at least insofar as using a minibatch of training examples to compute the gradient.

In other cases, the objective function we want to minimize is actually intractable. When the objective function is intractable, typically its gradient is intractable as well. In such cases we can only approximate the gradient. These issues mostly arise

with the more advanced models in part III. For example, contrastive divergence gives a technique for approximating the gradient of the intractable log-likelihood of a Boltzmann machine.

Various neural network optimization algorithms are designed to account for imperfections in the gradient estimate. One can also avoid the problem by choosing a surrogate loss function that is easier to approximate than the true loss.

8.2.7 Poor Correspondence between Local and Global Structure

Many of the problems we have discussed so far correspond to properties of the loss function at a single point—it can be difficult to make a single step if $J(\boldsymbol{\theta})$ is poorly conditioned at the current point $\boldsymbol{\theta}$, or if $\boldsymbol{\theta}$ lies on a cliff, or if $\boldsymbol{\theta}$ is a saddle point hiding the opportunity to make progress downhill from the gradient.

It is possible to overcome all of these problems at a single point and still perform poorly if the direction that results in the most improvement locally does not point toward distant regions of much lower cost.

Goodfellow *et al.* (2015) argue that much of the runtime of training is due to the length of the trajectory needed to arrive at the solution. Figure 8.2 shows that the learning trajectory spends most of its time tracing out a wide arc around a mountain-shaped structure.

Much of research into the difficulties of optimization has focused on whether training arrives at a global minimum, a local minimum, or a saddle point, but in practice neural networks do not arrive at a critical point of any kind. Figure 8.1 shows that neural networks often do not arrive at a region of small gradient. Indeed, such critical points do not even necessarily exist. For example, the loss function $-\log p(y \mid \mathbf{x}; \boldsymbol{\theta})$ can lack a global minimum point and instead asymptotically approach some value as the model becomes more confident. For a classifier with discrete y and $p(y \mid \mathbf{x})$ provided by a softmax, the negative log-likelihood can become arbitrarily close to zero if the model is able to correctly classify every example in the training set, but it is impossible to actually reach the value of zero. Likewise, a model of real values $p(y \mid \mathbf{x}) = \mathcal{N}(y; f(\boldsymbol{\theta}), \beta^{-1})$ can have negative log-likelihood that asymptotes to negative infinity—if $f(\boldsymbol{\theta})$ is able to correctly predict the value of all training set y targets, the learning algorithm will increase β without bound. See figure 8.4 for an example of a failure of local optimization to find a good cost function value even in the absence of any local minima or saddle points.

Future research will need to develop further understanding of the factors that influence the length of the learning trajectory and better characterize the outcome

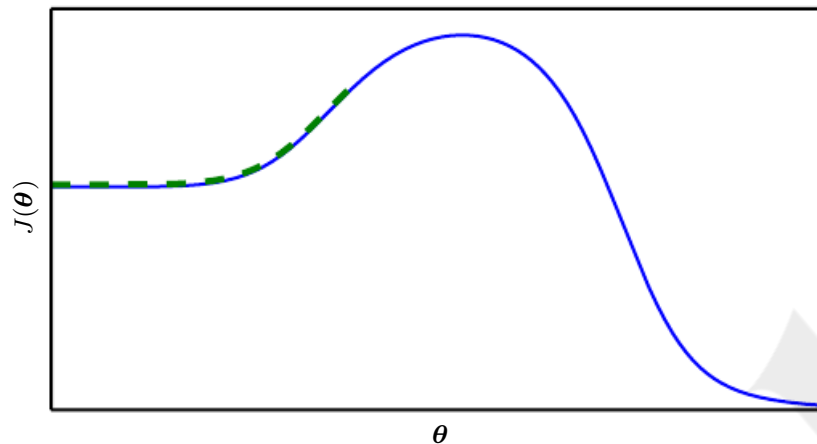


Figure 8.4: Optimization based on local downhill moves can fail if the local surface does not point toward the global solution. Here we provide an example of how this can occur, even if there are no saddle points and no local minima. This example cost function contains only asymptotes toward low values, not minima. The main cause of difficulty in this case is being initialized on the wrong side of the “mountain” and not being able to traverse it. In higher dimensional space, learning algorithms can often circumnavigate such mountains but the trajectory associated with doing so may be long and result in excessive training time, as illustrated in figure 8.2.

of the process.

Many existing research directions are aimed at finding good initial points for problems that have difficult global structure, rather than developing algorithms that use non-local moves.

Gradient descent and essentially all learning algorithms that are effective for training neural networks are based on making small, local moves. The previous sections have primarily focused on how the correct direction of these local moves can be difficult to compute. We may be able to compute some properties of the objective function, such as its gradient, only approximately, with bias or variance in our estimate of the correct direction. In these cases, local descent may or may not define a reasonably short path to a valid solution, but we are not actually able to follow the local descent path. The objective function may have issues such as poor conditioning or discontinuous gradients, causing the region where the gradient provides a good model of the objective function to be very small. In these cases, local descent with steps of size ϵ may define a reasonably short path to the solution, but we are only able to compute the local descent direction with steps of size $\delta \ll \epsilon$. In these cases, local descent may or may not define a path to the solution, but the path contains many steps, so following the path incurs a

high computational cost. Sometimes local information provides us no guide, when the function has a wide flat region, or if we manage to land exactly on a critical point (usually this latter scenario only happens to methods that solve explicitly for critical points, such as Newton’s method). In these cases, local descent does not define a path to a solution at all. In other cases, local moves can be too greedy and lead us along a path that moves downhill but away from any solution, as in figure 8.4, or along an unnecessarily long trajectory to the solution, as in figure 8.2. Currently, we do not understand which of these problems are most relevant to making neural network optimization difficult, and this is an active area of research.

Regardless of which of these problems are most significant, all of them might be avoided if there exists a region of space connected reasonably directly to a solution by a path that local descent can follow, and if we are able to initialize learning within that well-behaved region. This last view suggests research into choosing good initial points for traditional optimization algorithms to use.

8.2.8 Theoretical Limits of Optimization

Several theoretical results show that there are limits on the performance of any optimization algorithm we might design for neural networks (Blum and Rivest, 1992; Judd, 1989; Wolpert and MacReady, 1997). Typically these results have little bearing on the use of neural networks in practice.

Some theoretical results apply only to the case where the units of a neural network output discrete values. However, most neural network units output smoothly increasing values that make optimization via local search feasible. Some theoretical results show that there exist problem classes that are intractable, but it can be difficult to tell whether a particular problem falls into that class. Other results show that finding a solution for a network of a given size is intractable, but in practice we can find a solution easily by using a larger network for which many more parameter settings correspond to an acceptable solution. Moreover, in the context of neural network training, we usually do not care about finding the exact minimum of a function, but seek only to reduce its value sufficiently to obtain good generalization error. Theoretical analysis of whether an optimization algorithm can accomplish this goal is extremely difficult. Developing more realistic bounds on the performance of optimization algorithms therefore remains an important goal for machine learning research.